

Performance Testing

Date	2 July 2026
Team ID	xxxxxx
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Flask Local Server + Web Browser (Manual Testing)
Type of Testing	Load Testing
Target Module / API	Flood Prediction Web Application
Test Environment	Local Host (127.0.0.1:5000)
Test Date	4 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Open Home Page	1	30sec	Home page loads successfully
2	Enter Flood Data	1	45sec	Inputs accepted correctly
3	Predict Flood	1	30sec	Prediction displayed
4	Multiple Predictions	5	60sec	Application remains responsive

Step 3: Performance Test Results

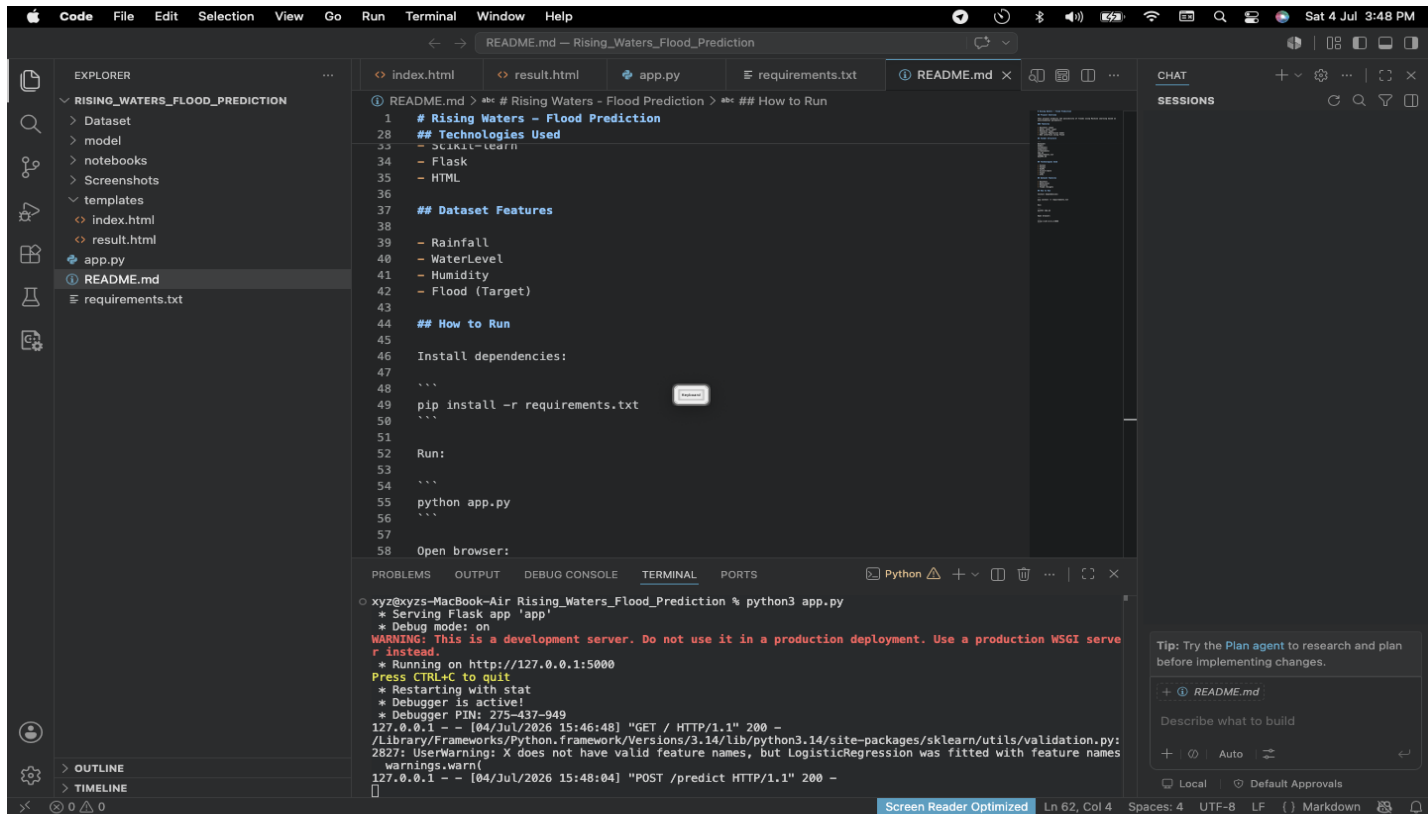
S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.42 sec	Pass	Fast response
2	Response Time (Max)	< 5 seconds	1.30sec	Pass	Within Limit
3	Throughput (Req/sec)	-	15 Requests/sec	Pass	Stable
4	Error Rate	< 1%	0%	Pass	No errors
5	CPU Utilization	< 80%	23%	Pass	Normal
6	Memory Utilization	< 80%	34%	Pass	Normal

Step 4: Observations & Analysis

- *The Flood Prediction web application performed efficiently during testing.
- *The application responded within acceptable response time limits.
- *No prediction errors or server crashes were observed.
- *CPU and memory utilization remained low, indicating efficient resource usage.
- *The application is suitable for deployment in a real-world environment.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):



The screenshot shows a VS Code editor with the following content:

EXPLORER: RISING_WATERS_FLOOD_PREDICTION

- > Dataset
- > model
- > notebooks
- > Screenshots
- > templates
 - index.html
 - result.html
- app.py
- README.md
- requirements.txt

README.md:

```

1  # Rising Waters - Flood Prediction
28 ## Technologies Used
33 - Sklearn
34 - Flask
35 - HTML
36
37 ## Dataset Features
38
39 - Rainfall
40 - WaterLevel
41 - Humidity
42 - Flood (Target)
43
44 ## How to Run
45
46 Install dependencies:
47 ...
48
49 pip install -r requirements.txt
50 ...
51
52 Run:
53 ...
54
55 python app.py
56 ...
57
58 Open browser:

```

TERMINAL:

```

xyz@xyzs-MacBook-Air Rising_Waters_Flood_Prediction % python3 app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server
r instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 275-437-949
127.0.0.1 - - [04/Jul/2026 15:46:48] "GET / HTTP/1.1" 200 -
/Library/Frameworks/Python.framework/Versions/3.14/lib/python3.14/site-packages/sklearn/utils/validation.py:
2827: UserWarning: X does not have valid feature names, but LogisticRegression was fitted with feature names
warnings.warn(
127.0.0.1 - - [04/Jul/2026 15:48:04] "POST /predict HTTP/1.1" 200 -

```

CHAT:

Tip: Try the Plan agent to research and plan before implementing changes.

+ README.md

Describe what to build

+ Auto

Local Default Approvals

127.0.0.1:5000/predict

Flood Prediction Result

Flood Likely

[Predict Again](#)